

```
# Convert the response object to a dictionary.
❺ response_dict = r.json()

# Process results.
print(response_dict.keys())
```

We first import the requests module. Then we assign the URL of the API call to the `url` variable ❶. This is a long URL, so we break it into two lines. The first line is the main part of the URL, and the second line is the query string. We've included one more condition to the original query string: `stars:>10000`, which tells GitHub to only look for Python repositories that have more than 10,000 stars. This should allow GitHub to return a complete, consistent set of results.

GitHub is currently on the third version of its API, so we define headers for the API call that ask explicitly to use this version of the API, and return the results in the JSON format ❷. Then we use requests to make the call to the API ❸. We call `get()` and pass it the URL and the header that we defined, and we assign the response object to the variable `r`.

The response object has an attribute called `status_code`, which tells us whether the request was successful. (A status code of 200 indicates a successful response.) We print the value of `status_code` so we can make sure the call went through successfully ❹. We asked the API to return the information in JSON format, so we use the `json()` method to convert the information to a Python dictionary ❺. We assign the resulting dictionary to `response_dict`.

Finally, we print the keys from `response_dict` and see the following output:

```
Status code: 200
dict_keys(['total_count', 'incomplete_results', 'items'])
```

Because the status code is 200, we know that the request was successful. The response dictionary contains only three keys: `'total_count'`, `'incomplete_results'`, and `'items'`. Let's take a look inside the response dictionary.

Working with the Response Dictionary

With the information from the API call represented as a dictionary, we can work with the data stored there. Let's generate some output that summarizes the information. This is a good way to make sure we received the information we expected, and to start examining the information we're interested in:

python
_repos.py

```
import requests

# Make an API call and store the response.
--snip--

# Convert the response object to a dictionary.
response_dict = r.json()
```